

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	RAJESH KUMAR V	Reg. No.:	192511389
Section / Batch:		Date:	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication	Q1

SECTION A – Experiments

Code of Conduct

I RAJESH KUMAR V / 192511389 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

SECTION A – Experiments (Answers)

Q1. Design and Code for Divide and Conquer Algorithms: Merge Sort, Quick Sort, Binary Search, and Matrix Multiplication. The algorithm, C program, sample execution/output, and complexity analysis for each experiment are given below.

Experiment 1: Merge Sort

Aim

To design and implement the Merge Sort algorithm using the Divide and Conquer technique and analyse its time complexity.

Algorithm

```
MERGE_SORT(A, low, high)
1. If low < high
    a. mid = (low + high) / 2
    b. MERGE_SORT(A, low, mid)           // Divide left half
    c. MERGE_SORT(A, mid+1, high)       // Divide right half
    d. MERGE(A, low, mid, high)         // Combine

MERGE(A, low, mid, high)
1. Copy A[low..mid] into L[] and A[mid+1..high] into R[]
2. Compare elements of L and R and place the smaller one back into A
3. Copy any remaining elements of L or R into A
```

C Program

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high) {
    int i, j, k, n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++) L[i] = a[low + i];
    for (j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    i = 0; j = 0; k = low;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else a[k++] = R[j++];
    }
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: "); scanf("%d", &n);
    int a[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    mergeSort(a, 0, n - 1);
    printf("Sorted array: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    return 0;
}
```

```
}
```

Sample Input / Output

```
Enter number of elements: 6
Enter 6 elements: 38 27 43 3 9 82
Sorted array: 3 9 27 38 43 82
```

Complexity Analysis

- Recurrence relation: $T(n) = 2T(n/2) + O(n)$
- Time Complexity (Best, Average, Worst): $O(n \log n)$
- Space Complexity: $O(n)$ - auxiliary arrays used during merge
- Merge Sort is stable and performance does not depend on initial data order.

Experiment 2: Quick Sort

Aim

To design and implement the Quick Sort algorithm using the Divide and Conquer technique and analyse its time complexity.

Algorithm

```
QUICK_SORT(A, low, high)
1. If low < high
    a. pivotIndex = PARTITION(A, low, high)
    b. QUICK_SORT(A, low, pivotIndex - 1)
    c. QUICK_SORT(A, pivotIndex + 1, high)

PARTITION(A, low, high)
1. pivot = A[high]; i = low - 1
2. For j = low to high-1: if A[j] <= pivot, i++, swap A[i], A[j]
3. Swap A[i+1], A[high]; return i+1
```

C Program

```
#include <stdio.h>
void swap(int *a, int *b) { int t = *a; *a = *b; *b = t; }

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n, i;
    printf("Enter number of elements: "); scanf("%d", &n);
    int a[n];
    printf("Enter %d elements: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    quickSort(a, 0, n - 1);
    printf("Sorted array: ");
```

```

    for (i = 0; i < n; i++) printf("%d ", a[i]);
    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 6
Enter 6 elements: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10

```

Complexity Analysis

- Best/Average Case: $O(n \log n)$ - pivot splits the array evenly
- Worst Case: $O(n^2)$ - already-sorted input with last-element pivot
- Space Complexity: $O(\log n)$ average (recursion stack); in-place
- Quick Sort is usually faster in practice than Merge Sort in-place.

Experiment 3: Binary Search

Aim

To design and implement the Binary Search algorithm using the Divide and Conquer technique and analyse its time complexity.

Algorithm

```

BINARY_SEARCH(A, low, high, key)
1. If low > high, return -1 (not found)
2. mid = (low + high) / 2
3. If A[mid] == key, return mid
4. If key < A[mid], BINARY_SEARCH(A, low, mid-1, key)
5. Else, BINARY_SEARCH(A, mid+1, high, key)

```

C Program

```

#include <stdio.h>
int binarySearch(int a[], int low, int high, int key) {
    if (low > high) return -1;
    int mid = (low + high) / 2;
    if (a[mid] == key) return mid;
    else if (key < a[mid]) return binarySearch(a, low, mid - 1, key);
    else return binarySearch(a, mid + 1, high, key);
}

int main() {
    int n, i, key, result;
    printf("Enter number of elements: "); scanf("%d", &n);
    int a[n];
    printf("Enter %d elements in sorted order: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter the key to search: "); scanf("%d", &key);
    result = binarySearch(a, 0, n - 1, key);
    if (result != -1) printf("Element found at index %d\n", result);
    else printf("Element not found\n");
    return 0;
}

```

Sample Input / Output

```

Enter number of elements: 7
Enter 7 elements in sorted order: 2 4 6 8 10 12 14
Enter the key to search: 10
Element found at index 4

```

Complexity Analysis

- Recurrence relation: $T(n) = T(n/2) + O(1)$

- Time Complexity: $O(\log n)$
- Space Complexity: $O(\log n)$ recursive, $O(1)$ iterative
- Binary Search requires the input array to be sorted beforehand.

Experiment 4: Matrix Multiplication (Divide and Conquer)

Aim

To design and implement Matrix Multiplication using the Divide and Conquer technique and analyse its time complexity.

Algorithm

```
MATRIX_MULTIPLY_DC(A, B, n)
1. If  $n == 1$ , return  $A[0][0] * B[0][0]$ 
2. Divide A, B into four  $(n/2 \times n/2)$  sub-matrices:
    $A = [A_{11} \ A_{12}; A_{21} \ A_{22}]$ ,  $B = [B_{11} \ B_{12}; B_{21} \ B_{22}]$ 
3. Compute recursively:
    $C_{11} = A_{11} * B_{11} + A_{12} * B_{21}$      $C_{12} = A_{11} * B_{12} + A_{12} * B_{22}$ 
    $C_{21} = A_{21} * B_{11} + A_{22} * B_{21}$      $C_{22} = A_{21} * B_{12} + A_{22} * B_{22}$ 
4. Combine  $C_{11}$ ,  $C_{12}$ ,  $C_{21}$ ,  $C_{22}$  into result matrix C
```

C Program

```
#include <stdio.h>
#define N 2
void multiply(int a[N][N], int b[N][N], int c[N][N]) {
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            c[i][j] = 0;
            for (int k = 0; k < N; k++) c[i][j] += a[i][k] * b[k][j];
        }
}

int main() {
    int a[N][N], b[N][N], c[N][N];
    printf("Enter elements of matrix A (2x2): ");
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) scanf("%d", &a[i][j]);
    printf("Enter elements of matrix B (2x2): ");
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) scanf("%d", &b[i][j]);
    multiply(a, b, c);
    printf("Resultant Matrix C = A x B:\n");
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}
```

Sample Input / Output

```
Enter elements of matrix A (2x2): 1 2 3 4
Enter elements of matrix B (2x2): 5 6 7 8
Resultant Matrix C = A x B:
19 22
43 50
```

Complexity Analysis

- Standard D&C recurrence: $T(n) = 8T(n/2) + O(n^2) \rightarrow O(n^3)$
- Strassen's method: $T(n) = 7T(n/2) + O(n^2) \rightarrow O(n^{2.81})$
- Space Complexity: $O(n^2)$ for storing sub-matrices
- D&C matrix multiplication benefits large matrices; overhead dominates for small n .

RUBRICS FOR CALCULATION

Experiments					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding ; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____